

routeDataFetchService.ts — Line-by-Line Reference

File Purpose

This service fetches route statistics from the backend API and returns them as rows for display in a stats table. It supports two query modes: **date range** and **session ID**.

Line-by-Line Breakdown

Line 1 — API Base URL

```
const API_BASE = process.env.NEXT_PUBLIC_API_URL ?? "";
```

Reads the backend URL from the `NEXT_PUBLIC_API_URL` environment variable. Falls back to `""` (empty string), meaning requests will use relative paths if the variable is not set.

Lines 3–6 — `RouteStatRow` Type

```
export type RouteStatRow = {
  metric: string;
  value: number | string;
};
```

The shape of a single stat row returned to the UI. `metric` is the human-readable label (e.g., "Total Routes Completed"), and `value` is the result from the API — either a number or a string like `"--"` or `"Error"`.

Lines 8–19 — `DATE_RANGE_ENDPOINTS`

```
const DATE_RANGE_ENDPOINTS: { endpoint: string; label: string }[] = [ ... ];
```

An array of 10 backend endpoints used when querying by **date range**. Each entry pairs:

- `endpoint` — the API route path (e.g., `/api/route-stats/total-routes-completed-by-date-range`)
- `label` — the display name shown in the stats table

Label	Endpoint
Total Routes Completed	<code>/api/route-stats/total-routes-completed-by-date-range</code>
Total Connected Users	<code>/api/route-stats/total-connected-users-by-date-range</code>
Users Started More Than One Route	<code>/api/route-stats/users-started-more-than-one-route-by-date-range</code>
Users Completed More Than One Route	<code>/api/route-stats/users-completed-more-than-one-route-by-date-range</code>

Users Never Started Route	/api/route-stats/users-never-started-route-by-date-range
Users Started But Not Finished	/api/route-stats/users-started-but-not-finished-by-date-range
Total Replays	/api/route-stats/total-replays-by-date-range
Avg Routes Per User	/api/route-stats/avg-routes-per-user-by-date-range
Avg Routes Per Session	/api/route-stats/avg-routes-per-session-by-date-range
Replay Ratio	/api/route-stats/replay-ratio-by-date-range

Lines 21–32 — SESSION_ENDPOINTS

```
const SESSION_ENDPOINTS: { endpoint: string; label: string }[] = [ ... ];
```

Identical structure to `DATE_RANGE_ENDPOINTS`, but the paths use `-by-session` suffixes. These endpoints accept a `sessionId` instead of a date range. The 10 metrics are the same.

Lines 34–55 — fetchStat (private helper)

```
async function fetchStat(endpoint: string, body: object): Promise<number | string>
```

A generic internal helper that POSTs to a single endpoint and returns the numeric or string result.

Step-by-step:

Lines	What happens
35–39	<code>fetch</code> is called with <code>POST</code> , <code>Content-Type: application/json</code> , and the serialized body
40–41	The raw response content-type header and body text are read
42–44	If the response is not JSON (e.g., an HTML error page), throws "Server returned an invalid response."
45–50	Attempts to <code>JSON.parse</code> the text; throws "Invalid response from server." on failure
51–53	If the HTTP status is not OK or <code>data.success</code> is falsy, throws using <code>data.message</code> or <code>data.error</code>
54	Returns <code>data.value</code> if present, otherwise the fallback string <code>"--"</code>

Lines 57–69 — fetchRouteStatsByDateRange

```
export async function fetchRouteStatsByDateRange(startDate: string, endDate: string): Promise<RouteStatRow[]>
```

Public function called by the UI to fetch all 10 date-range stats in **parallel**.

- **Line 58** — builds the POST body `{ startDate, endDate }`
- **Lines 59–68** — maps every entry in `DATE_RANGE_ENDPOINTS` to a `fetchStat` call using `Promise.all`, so all 10 requests fire simultaneously
- **Lines 62–65** — each individual call is wrapped in `try/catch`; a failed request returns `{ metric: label, value: "Error" }` instead of crashing the whole fetch

Lines 71–83 — `fetchRouteStatsBySession`

```
export async function fetchRouteStatsBySession(sessionId: string):  
  Promise<RouteStatRow[]>
```

Public function for fetching all 10 session stats in **parallel**.

- **Line 72** — builds the POST body `{ sessionId: Number(sessionId) }` — the string param is coerced to a number here
- **Lines 73–82** — same `Promise.all` + per-item `try/catch` pattern as the date-range variant, using `SESSION_ENDPOINTS`

Summary

Export	Input	What it fetches
RouteStatRow	(type)	Shape of one stats table row
fetchRouteStatsByDateRange	startDate, endDate	10 metrics filtered by date range
fetchRouteStatsBySession	sessionId	10 metrics filtered by session

All fetches run in parallel per query. Individual failures surface as `"Error"` in that row rather than failing the entire table load.